RAG Chunking Test Document

Page 1 - Company Policy

BlueNova Analytics is a fictional software company used for testing this document question-answering project. The company has 120 employees working across engineering, product, sales, finance, and customer support. The main office is located in Lyon, France, but many employees work remotely from other cities. The standard work week is thirty-five hours. Employees can organize their schedule between 8:00 and 19:00 as long as they attend required team meetings and complete their responsibilities.

Every full-time employee receives twenty-five paid vacation days per calendar year. In addition, employees receive the public holidays that apply in their work location. Vacation requests of one or two days should normally be submitted at least three business days in advance. Requests for five consecutive working days or more should normally be submitted at least two weeks in advance. Managers may approve exceptions when there is an urgent personal situation.

The remote-work policy allows employees to work from home up to three days per week. Tuesday is the recommended in-office collaboration day for the engineering department. Employees who need a fully remote arrangement must obtain written approval from their manager and Human Resources. The company provides a laptop, an external monitor, and a headset to permanent employees. Equipment remains company property and must be returned when employment ends.

For internal communication, BlueNova uses Microsoft Teams for chat and meetings. Jira is used to track development work and defects. Confluence is used for team documentation. Git repositories are hosted in Bitbucket. Employees should never store company passwords in plain text documents or source-code repositories. Passwords and API keys must be kept in approved secret-management systems.

Page 2 - AI Document Q&A; Architecture

The AI Document Q&A; application accepts PDF and TXT documents. When a user uploads a document, the backend first validates the file type. PDF files are read with a document parser that extracts text page by page. TXT files are decoded as text. The extracted content is then converted into a common structure that contains the page number, when a page number exists, and the text content.

After extraction, the application splits the text into smaller pieces called chunks. The first experimental configuration uses a chunk size of two hundred words and an overlap of fifty words. Overlap means that the end of one chunk is repeated at the beginning of the next chunk. This helps preserve context when an important sentence or idea is located near a chunk boundary. Very small trailing chunks may be skipped or merged because a one-word or two-word chunk is usually not useful for retrieval.

Each chunk is converted into an embedding. An embedding is a numerical vector that represents the meaning of the text. Texts with similar meanings should have vectors that are relatively close to each other. The vector for a user question can therefore be compared with the vectors for document chunks. The most similar chunks are retrieved and used as context for the language model.

The planned backend framework is FastAPI. PostgreSQL is the main relational database, and pgvector can be used to store and search embedding vectors. The frontend can be implemented with React. The application should return answers that are grounded in retrieved document text. If the document does not contain enough information to answer a question, the system should say that the answer could not be found instead of inventing information.

Page 3 - Security, Testing, and Evaluation

The application must reject unsupported file types and handle malformed documents safely. API keys and other secrets must be stored in environment variables and must not be committed to Git. The system should separate application instructions from retrieved document content so that text inside an uploaded document cannot easily override the system rules. Logging should capture useful technical events without unnecessarily recording sensitive document content.

Testing should cover both normal and failure scenarios. Normal scenarios include uploading a valid PDF, uploading a valid TXT file, extracting text, creating chunks, retrieving relevant chunks, and generating an answer with citations. Failure scenarios include unsupported file types, empty documents, malformed PDFs, missing information, irrelevant retrieval results, and adversarial instructions embedded inside a document.

Evaluation should not rely only on a successful demonstration. A small benchmark should contain documents, questions, and manually verified answers. Retrieval quality can be measured by checking whether the relevant chunk appears in the top results. Groundedness checks whether the answer is supported by the retrieved text. Answer correctness checks whether the response answers the question. Citation accuracy checks whether the cited page or chunk actually contains the evidence used in the answer.

Example questions for this test document include: How many vacation days do employees receive? Which day is recommended for engineering collaboration in the office? Which tool hosts Git repositories? What chunk size and overlap are used in the first experiment? What database technology is planned for vector search? What should the system do when the answer is missing from the document? These questions are intentionally spread across different sections so that retrieval can be tested later.